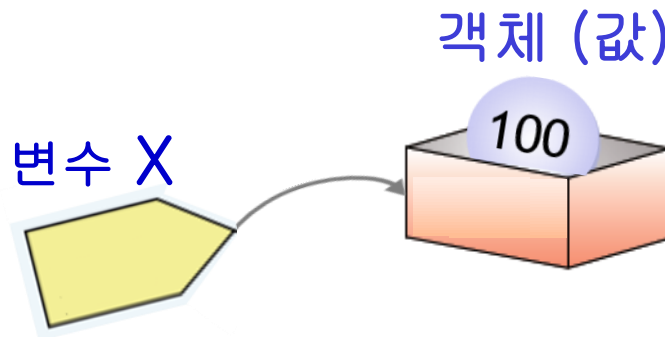


CHAP 2: 변수와 자료형

변수(variable), 변수 할당문,
변수 사용, 여러 변수 할당문,
변수 이름,
데이터 형식 (data types),
자료형 변환

변수(variable)

- 변수란?
어떤 값에 이름표를 붙여둔 것과 비슷
- 컴퓨터에서 처리되는 데이터는 메모리에 존재한다.
- 변수는 메모리에 있는 값(Value)을 참조하는 이름.
- 다른 값을 참조할 수도 있기 때문에 “variable(변수)”라 한다.
 - 변수는 실행 도중 다른 값을 할당 받을 수 있다.



간단히 “x 값은 100이다” 또는
“x는 100이다” 라고 한다.

변수 할당문

- 변수 만들기? 할당문(Assign Statement)을 사용.

변수 = 값

- 파이썬에서는 변수에 값이 할당되면 변수가 생성됨.
- 할당 연산자 = 를 사용.
- 할당 연산자의 왼쪽에는 생성하려는 변수를 지정,
오른쪽에는 할당하려는 값을 지정.
- 오른쪽 값에는 무엇이든 (값, 변수, 수식, 함수 등등) 올 수 있다.

```
>>> x = 100    # 값.           x는 100
>>> y1 = x     # 변수.         y1는 100
>>> y2 = 2+3   # 산술계산식.  y2는 5
>>> z = float(10) # 함수.      z는 10.0
>>>
```

변수 할당문

- (예) $x = 100$

- 100이라는 값은 메모리 어딘가에 위치,
변수 x 가 그 값을 가리키는 것



- 등호 표시 =

- 수학에선 '같음'을 의미
- 파이썬에서는 바인딩(binding)을 의미. (바인딩은 '가리키는 것')

파이썬에서 =



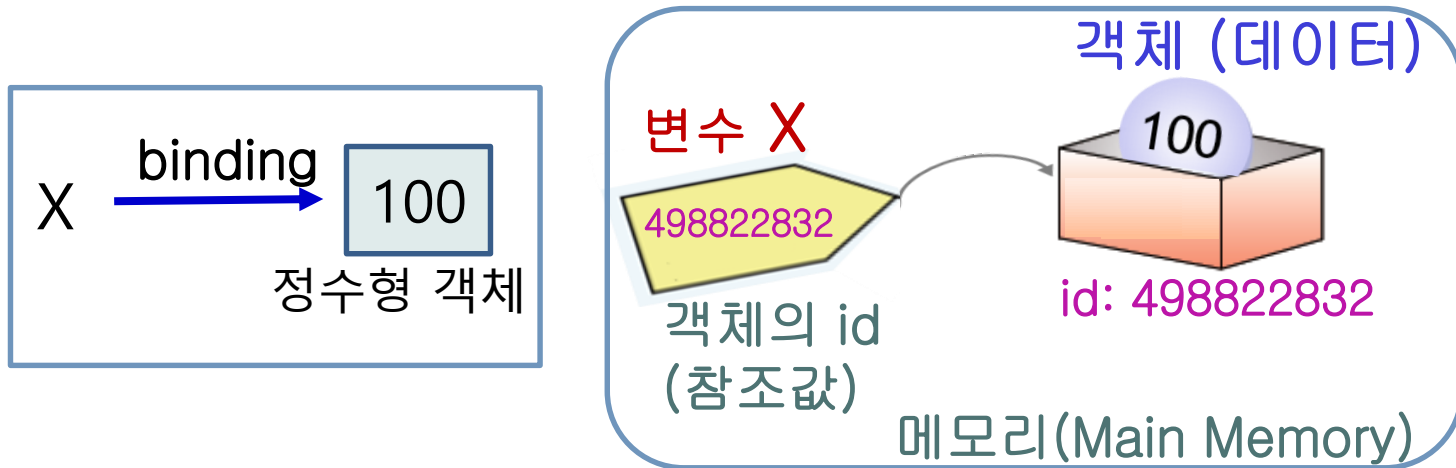
변수에 값을 할당하라는 의미.
(양변이 같다는 의미가 아님)

변수 할당문

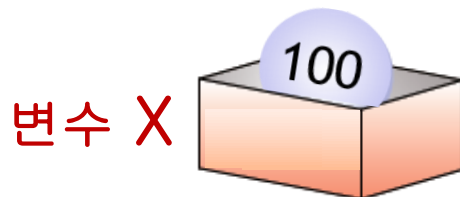
- 코드를 실행했을 때 내부에서 어떤 일이 벌어질까요?

```
>>> x = 100 Enter
```

- 먼저 100이라는 값(객체)이 메모리에 생성됨.
- 변수 x 에는 객체 100의 참조값(주소)이 저장된다.
- 즉, 변수 x는 객체 100을 가리키는 이름표. 값 100을 가리킴.

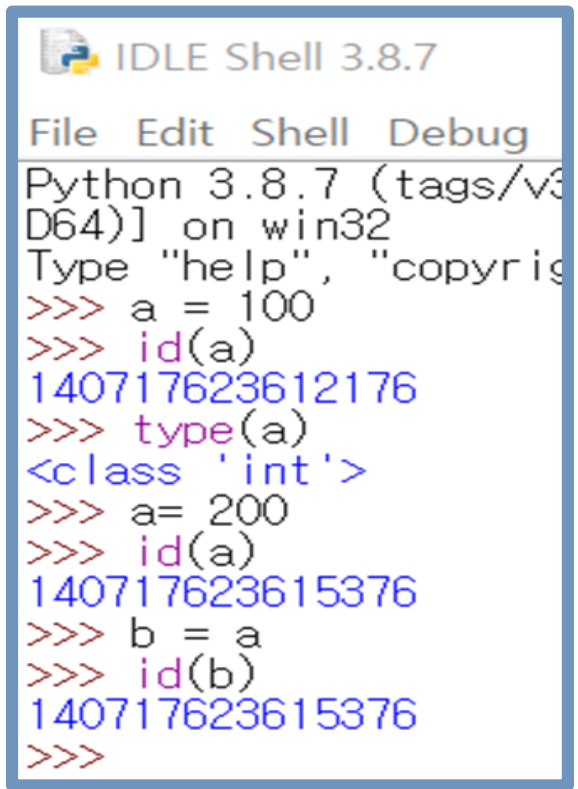


- 비교.** 다른 언어에서는, 변수 안에 값이 저장



변수 할당문

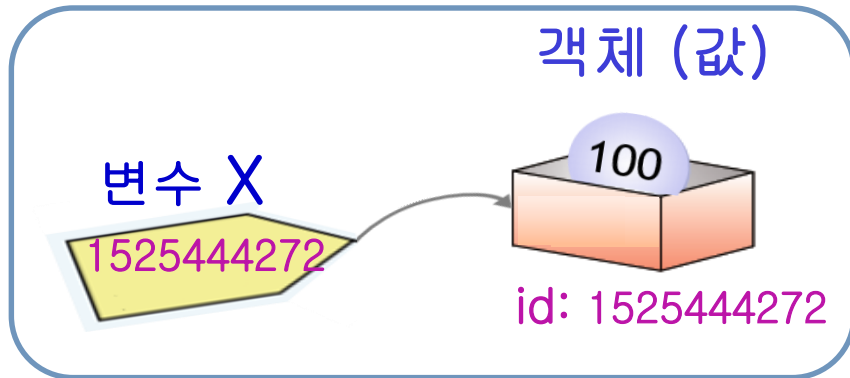
- 모든 데이터(값)는 객체, 메모리에 존재.
- 객체: 메모리에 존재하는 개별 데이터.
 - 객체의 형태:
 - `type()` 함수로 확인가능.
 - 객체의 `id`:
 - 객체를 다른 객체와 구별하는 고유번호. 즉, 참조 값. 메모리 상의 위치.
- 변수는
 - 값이 할당되는 순간에 생성됨. (변수선언 없음)
 - 메모리에 생성된 객체(object)를 참조하기 위한 수단.
 - 객체에 붙는 **이름**. 객체에 **여러 개의 이름**을 붙일 수도 있다.
 - 할당된 값을 꺼내려면 변수의 이름을 적어준다.
 - 변수에 다른 형식의 데이터를 다시 할당할 수 있다
 - **변수의 데이터 형식은 값이 할당되는 순간마다 변경된다.**

A screenshot of the IDLE Shell 3.8.7 window. The title bar says 'IDLE Shell 3.8.7'. The menu bar includes 'File', 'Edit', 'Shell', and 'Debug'. The shell shows the following text: 'Python 3.8.7 (tags/v3.8.7:6448b86, Sep 14 2019) on win32', 'Type "help", "copyright", and "credits()" for more', '>>> a = 100', '>>> id(a)', '140717623612176', '>>> type(a)', '<class \'int\'>', '>>> a = 200', '>>> id(a)', '140717623615376', '>>> b = a', '>>> id(b)', '140717623615376', and '>>>' at the end.

```
IDLE Shell 3.8.7
File Edit Shell Debug
Python 3.8.7 (tags/v3.8.7:6448b86, Sep 14 2019) on win32
Type "help", "copyright", and "credits()" for more
>>> a = 100
>>> id(a)
140717623612176
>>> type(a)
<class 'int'>
>>> a = 200
>>> id(a)
140717623615376
>>> b = a
>>> id(b)
140717623615376
>>>
```

변수 할당문

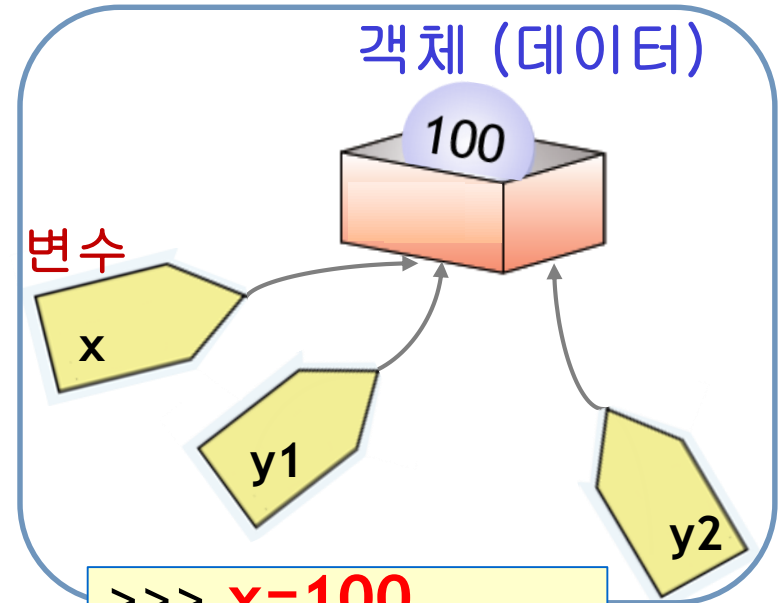
메모리(Main Memory)



```
>>> x=100   
>>> type(x)   
<class 'int'>  
>>> id(x)   
1525444272  
>>> X   
100  
>>>
```

Python Shell 창

메모리(Main Memory)



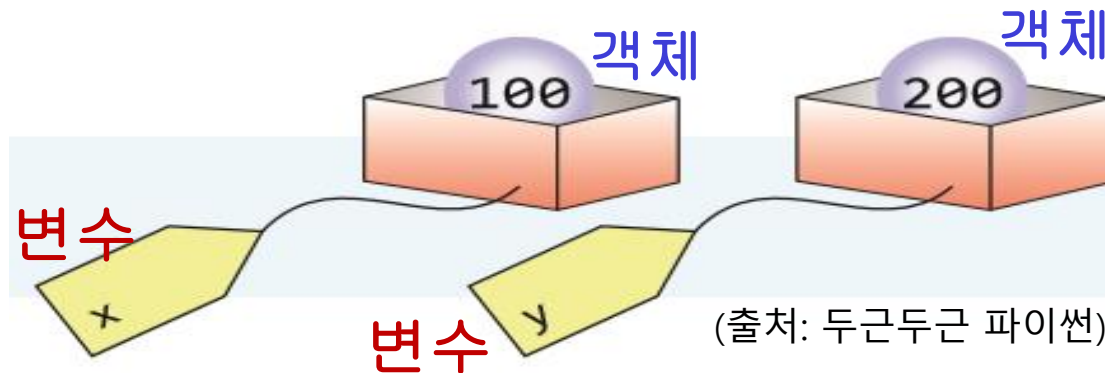
```
>>> x=100  
>>> x  
100  
>>> y1= x  
>>> y1  
100  
>>> y2= y1  
>>> y2  
100  
>>>
```

변수 사용

- 변수 2개 생성

```
>>> x = 100  
>>> y = 200
```

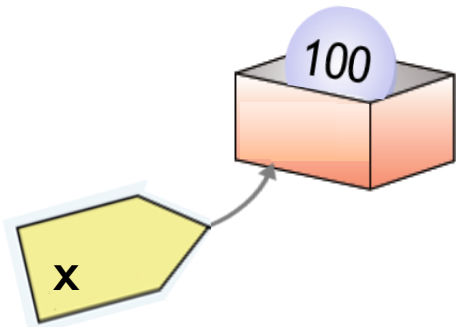
- 정수형 object 100과 이 객체를 가리키는 variable x 생성.
 - 간단히 “x 값은 100이다” 또는 “x는 100이다” 라고 한다.
- 정수형 object 200과 이 객체를 가리키는 variable y 생성.
 - 간단히 “y 값은 200이다” 또는 “y는 200이다”라고 한다.



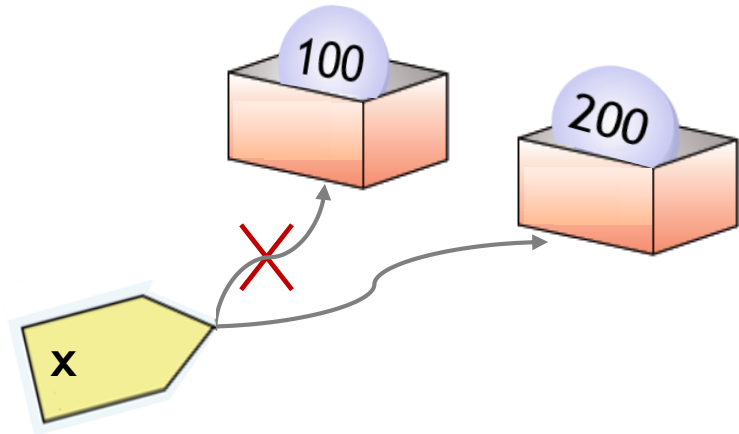
변수 사용

- 생성된 변수에 다른 값을 할당할 수 있다.
 - 변수 **x**가 정수 **100**을 가리키고 있는데, 정수 **200**을 가리키도록 변경한 것.

```
>>> x = 100  #(1)
>>> x = 200  #(2)
>>> print(x)
200
>>>
```



객체 100을 할당받음.



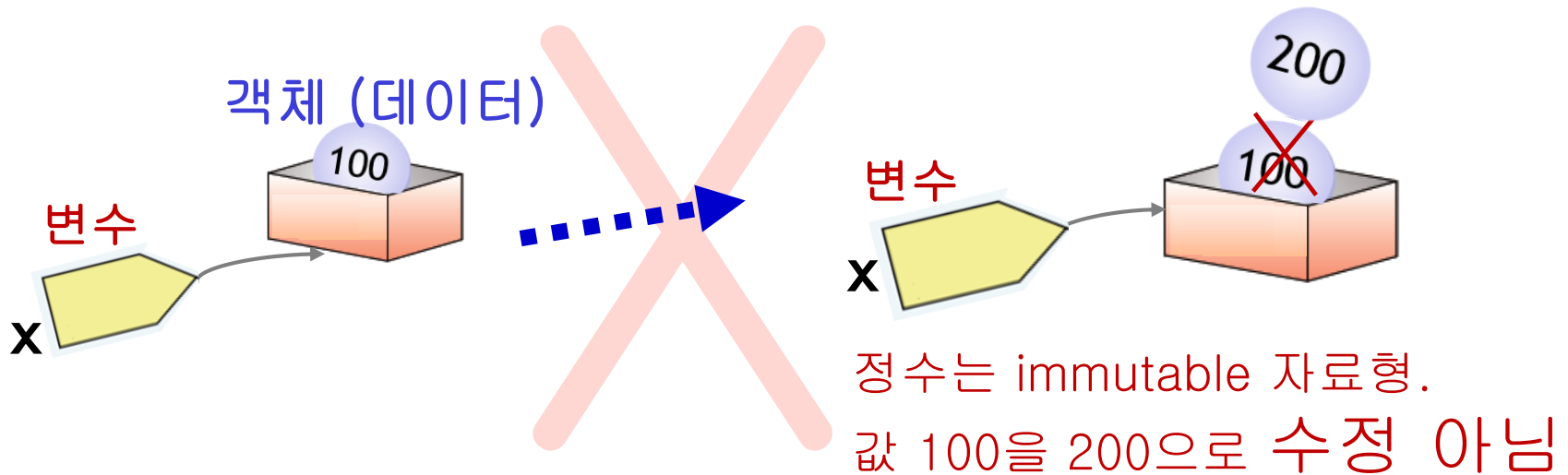
정수는 immutable 자료형.
따라서, 객체 200을 새로 할당받음.
값 100을 200으로 수정하는 것 아님

변수 사용

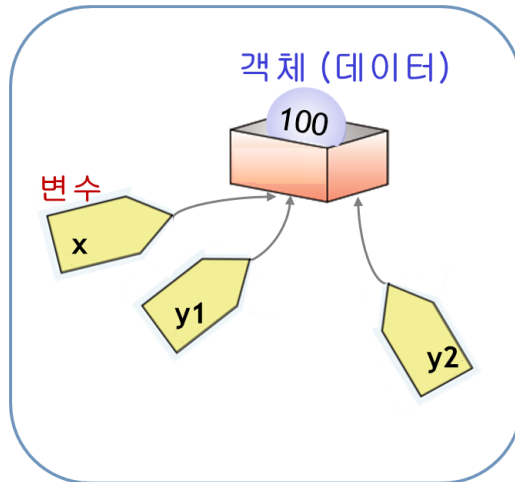
- 값 100을 200으로 수정하는 것 아님!!!
정수는 immutable 자료형(수정할 수 없는 자료형)

```
>>> x=100 # (1)
>>> x=200 # (2)
>>> print(x)
200
>>>
```

(2)실행 후



변수 사용



메모리(Main Memory)

```
>>> x=100
```

```
>>> x
```

```
100
```

```
>>> y1= x
```

```
>>> y1
```

```
100
```

```
>>> y2= y1
```

```
>>> y2
```

```
100
```

```
>>> x=200
```

```
>>> x
```

```
200
```

```
>>> y1
```

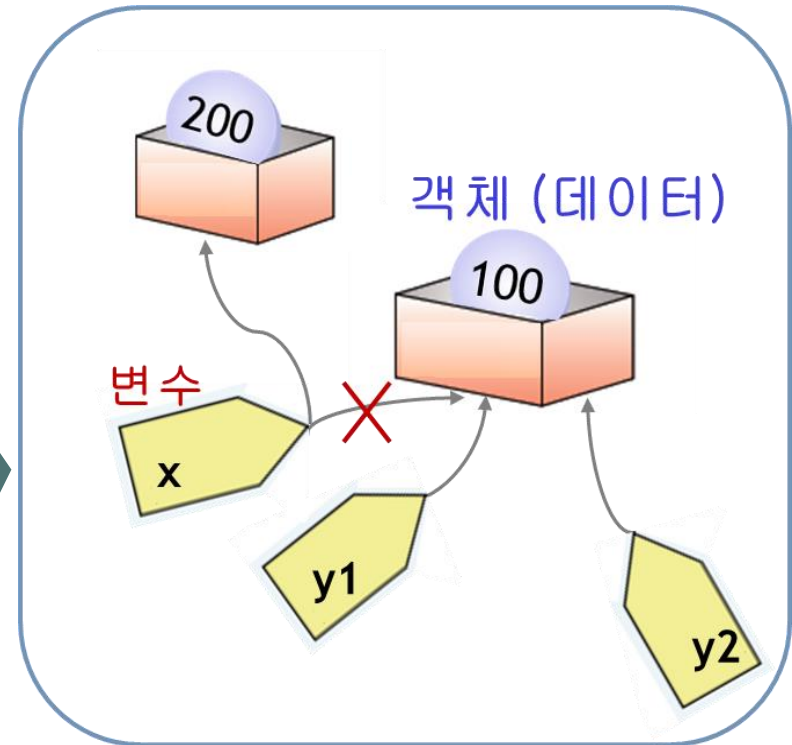
```
100
```

```
>>> y2
```

```
100
```

```
>>>
```

Python Shell 창



메모리(Main Memory)

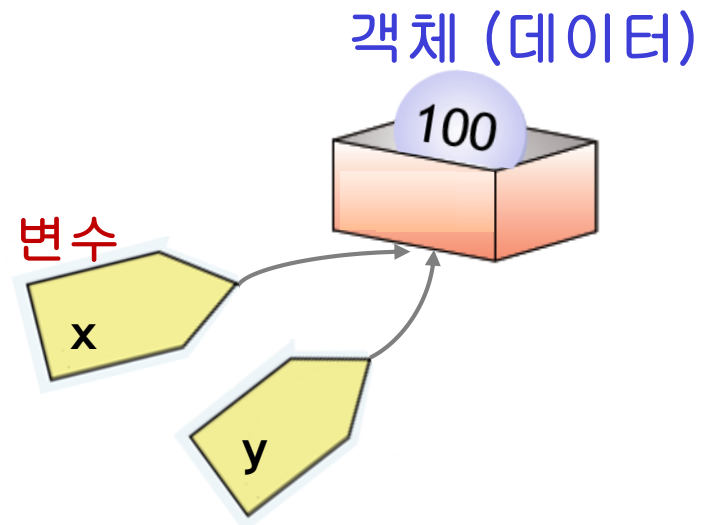
변수 사용

- 다른 변수의 값을 변수에 할당하기

(예)

```
x = 100  
y = x
```

- $x = 100$
정수형 object 100과 이 객체를 가리키는 variable x를 생성.
- $y = x$
x가 가리키는 object의 참조 값으로 variable y를 생성.



변수 사용

- 숫자끼리의 계산 결과를 변수에 할당하기

```
var1 = 100+100 # var1=200
```

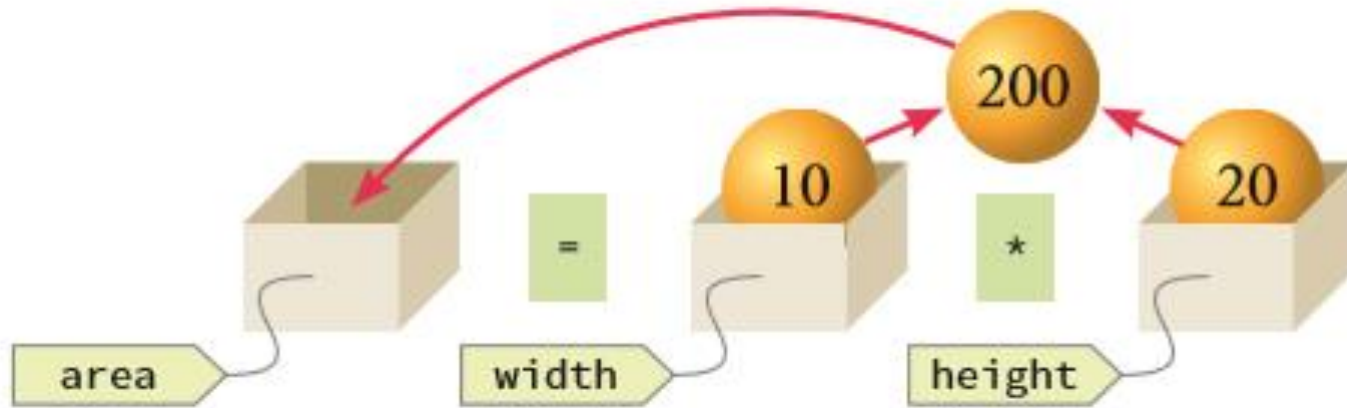
- 변수와 숫자의 연산(계산 결과)을 변수에 할당하기

```
var2 = var1+100 # var2=300
```

변수 사용

- 예) 면적 계산하기. 결과를 변수에 할당

```
>>> width = 10   
>>> height = 20   
>>> area = width * height   
>>> print(area)   
200
```



(출처: 어서와 파이썬은 처음이지)

변수 사용

- 변수의 값을 연산에 사용하고, 그 결과를 변수에 다시 할당하기.

```
var1 = 100
```

```
var1 = var1+200
```

변수 **var1**을 연산에 사용하고,
결과를 변수 **var1**에 새로 할당.
var1 += 200 과 동일.

- 연산과 할당을 동시에 표현

```
var1 = var1 + 200
```



```
var1 = var1 + 200
```



```
var1 += 200
```

var1에 300 할당.

변수 사용

- 변수에는 **문자열**도 할당할 수 있다.
 - **큰따옴표**(")로 양쪽을 둘러싸거나 **작은 따옴표**(')로 양쪽 둘러싼다.

```
>>> name = "홍길동 " 
```

```
>>> name 
```

‘홍길동’

```
>>> address = "서울시 마포구 신수동 " 
```

```
>>> address 
```

‘서울시 마포구 신수동’

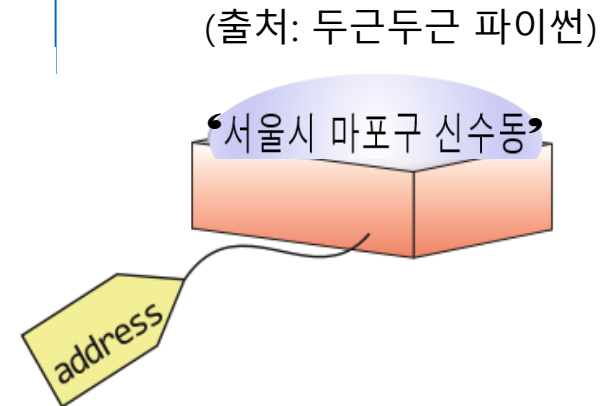
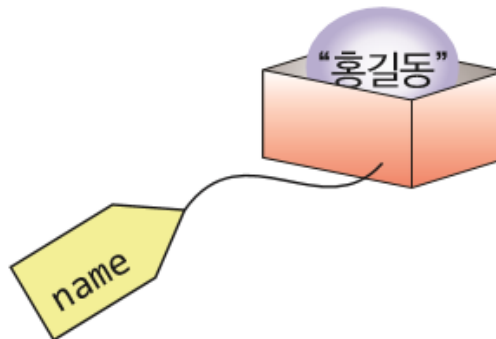
```
>>> print(name) 
```

홍길동

```
>>> print(address) 
```

서울시 마포구 신수동

```
>>>
```



여러 변수 할당문

- 여러 변수에 각각 값을 동시에 할당.(동시 할당문)
 - 한 문장으로, 여러 변수에 각각 값을 동시에 할당
 - 할당연산자를 기준으로 변수와 값의 개수가 같아야 함
변수1, 변수2, ..., 변수n = 값1, 값2, ..., 값n
- 예) boolVar, intVar, floatVar, strVar = True, 0, 0.0, "Hello!"



```
boolVar = True  
intVar = 0  
floatVar = 0.0  
strVar = "Hello!"
```

```
>>> boolVar, intVar, floatVar, strVar = True, 0, 0.0, "Hello!"  
>>> print(boolVar, intVar, floatVar, strVar)  
True 0 0.0 Hello!  
>>>
```

여러 변수 할당문

- 예) num1과 num2 값을 서로 바꾸기

```
>>> num1= 100  
>>> num2= 200
```

- 방법1 # 동시 할당문 사용

```
num1, num2 = num2, num1
```

```
>>> num1, num2 = num2, num1  
>>>  
>>> num1 # num1 값  
200  
>>> num2 # num2 값  
100  
>>>
```

- 방법2

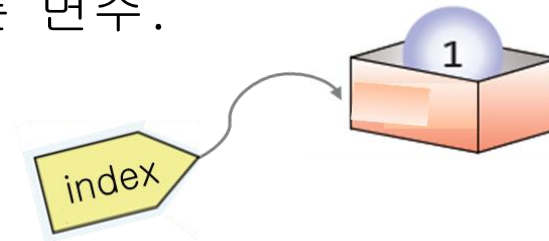
```
temp = num1  
num1 = num2  
num2 = temp
```

```
>>> temp = num1  
>>> num1 = num2  
>>> num2 = temp  
>>>  
>>> num1 # num1 값  
200  
>>> num2 # num2 값  
100  
>>>
```

변수 이름

- 변수명 (Naming) 규칙

- 변수의 이름은 문자, 숫자, 밑줄(_)로만 이루어짐.
 - 다른 기호를 사용하면 구문 에러(Syntax Error).
 - 변수의 이름 중간에 공백이 들어갈 수 없다.
 - (예) Money\$: 구문 에러, \$는 사용할 수 없다.
- 변수명은 문자 또는 밑줄로만 시작해야 한다.
 - 숫자로 시작 하면 안된다.
 - (예) 7up, 5brothers : 숫자로 시작했기 때문에 구문 에러.
- 파이썬 지정단어 (Keyword, Reserved word)들은 변수명으로 사용할 수 없다.
- 파이썬에서는 대문자와 소문자를 구분한다.
 - (예) hour 와 Hour는 다른 변수.



변수 이름

- 파이썬 지정단어 (Keyword/ Reserve Word)
 - 변수명으로 사용 할 수 없음
 - 파이썬 지정단어들의 목록 확인.
 - keyword 모듈을 import하고,
이 모듈 안에 있는 변수 kwlist 의 값을 확인해 본다.
 - keyword.kwlist

```
>>> import keyword
>>> keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert',
'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except',
'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda',
'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with',
'yield']
>>>
```

변수 이름

- 사용 가능한 변수 이름의 예

sum_	# 영문 알파벳 문자로 시작
_count	# 밑줄 문자로 시작할 수 있다.
number_of_pictures	# 중간에 밑줄 문자를 넣을 수 있다.
King3	# 첫 글자가 아니라면 숫자가 올 수 있다.

tutorName	# 영문자로구성
tutorname1	# 첫 글자가 아닐 경우 숫자 사용 가능
_tutorname	# 밑줄 문자로 시작 가능
tutor_name	# 중간에 밑줄 문자 사용 가능
TutorName	# 대/소문자가 구분되므로
	# TutorName과 tutorName은 서로 다른 변수
if_tutor	# 예약어 if가 다른 글자에 붙여 함께 사용 가능

변수 이름

- 잘못된 식별자 (변수 이름)

2nd_base	(X)	# 숫자로 시작할 수 없다.
money#	(X)	# #과 같은 기호는 사용할 수 없다.

varname\$	# 특수 문자를 사용할 수 없음
var name	# 중간에 공백을 사용할 수 없음
1varname	# 숫자로 시작할 수 없음
for	# 예약어 for를 단독으로 변수명으로 사용할 수 없음

데이터 형식 (Data Types)

- 변수에는 정수, 실수, 문자열 등의 다양한 값들을 할당할 수 있다.
- 파이썬의 기본 자료형(class)

자료형	예
정수 <class 'int'>	7, 123, -256
실수 <class 'float'>	3.14, -1.2345, -3.0e5
부울형(boolean) <class 'bool'>	True, False
문자열 <class 'str'>	"Hello", 'Hello'
리스트 <class 'list'>	[1, 2.2, "Hello"]
튜플 <class 'tuple'>	(1, 2.2, "Hello")
집합 <class 'set'>	{1, 2.2, 'Hello'}
사전 <class 'dict'>	{'키':170, '체중':62}

차후 설명

데이터 형식

- 변수에 어떤 종류의 데이터도 다시 할당할 수 있다
 - `type(변수)` 내장함수. 변수의 자료형을 알려주는 함수.
- (예) 변수 `x`에 다양한 자료형 값을 할당하고, 자료형을 확인해 보자

```
>>> x = 10
>>> type(x) # 변수 x가 할당 받은 값의 자료형(data type) 확인
<class 'int'>
>>> x = 2.5
>>> type(x) # 변수 x의 자료형 확인. float(실수형)으로 변경
<class 'float'>
>>> x = "Hello"
>>> type(x) # 변수 x의 자료형 확인. str(문자열형)으로 변경
<class 'str'>
>>> x=True
>>> type(x) # 변수 x의 자료형 확인. Boolean형으로 변경
<class 'bool'>
>>>
```


데이터 형식

- 정수형 (int)
 - 정수형은 소수점이 없는 데이터. (예) 100, -123, 0
 - int는 정수 데이터 형식
 - 파이썬에서 정수의 크기에 제한이 없다.
- 실수형 (float)
 - 실수형은 소수점이 있는 데이터. (예) 3.14, -2.7
 - 8바이트를 사용해서 근사치를 표현. 따라서, 오차발생. 실수를 정확히 표현할 수 없는 문제가 있다.
 - 사칙연산인 +, -, *, /를 수행

```
>>> 0.1 + 0.1 + 0.1    # 정확히 0.3이 아님
0.30000000000000004
>>> 0.1 + 0.1 + 0.1 == 0.3
False
>>>
```

데이터 형식

- `round()`를 이용해 실수를 반올림 해줌으로써 오차를 해결할 수 있다.
- `round(실수)` 또는 `round(실수, 자릿수)`
 - 실수를 반올림하는 함수
 - 자릿수: 반올림 후의 소수점 이하 자릿수
 - 자릿수를 지정하지 않으면, 반올림 후 정수 반환

```
>>> num= (26+54+96)/3
>>> num
58.666666666666664
>>> y= round(num) #반올림하여 정수 반환
>>> y
59
>>> print( y )
59
>>> print( round(num, 2) ) # 반올림 후 소수점 이하 자릿수 2
58.67
>>>
```

데이터 형식

```
>>> round(0.1+0.1+0.1, 1) # 반올림 후 소수점 이하 자릿수 1
0.3
>>>
>>> round(0.1 + 0.1 + 0.1, 1) == 0.3
True
>>>
```

- 부울형 (Boolean type)
 - **True** 또는 **False**
 - 참(True)이나 거짓(False)을 나타냄
 - 부울형은 단독으로 사용하기보다
if 조건문 이나 while 반복문 등과 함께 사용

데이터 형식

- 100과 “100”의 차이
 - 100 -> 정수
 - “100”, ‘100’ -> 문자열

```
>>> print(100+200)
300
>>> print("100"+"200")
100200
>>> 100+200
300
>>> '100' + '200'
'100200'
```

```
>>> number= 100 # 정수
>>> number
100
>>> s= "100" # 문자열
>>> s # 할당된 문자열을 보여줌
'100'
>>> print(s) # 변수 s의 값을
100 # 출력
```

“100”+“200”은 텍스트와 텍스트끼리 합하는
것이기 때문에 그냥 2개의 텍스트가 연결됨.

자료형 변환

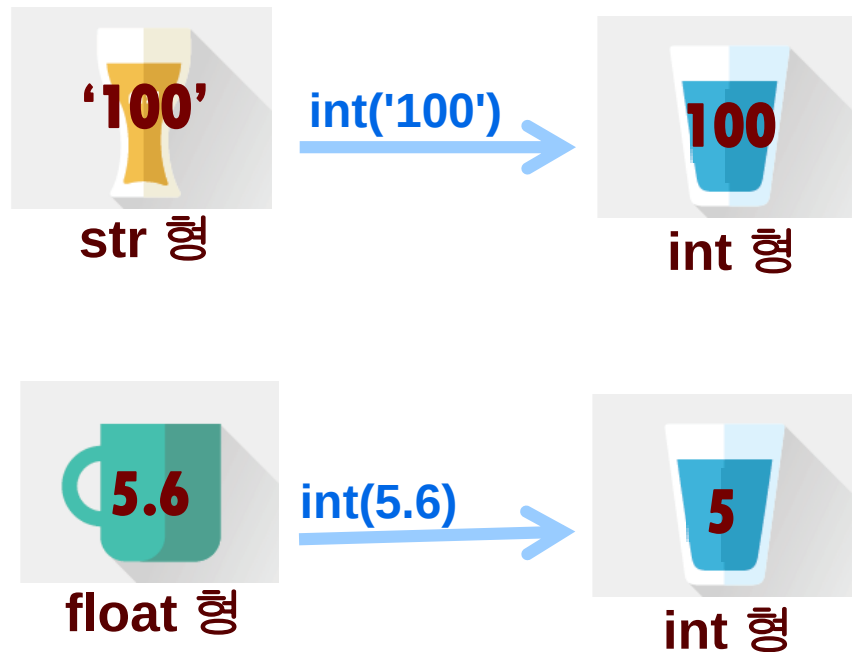
- 정수, 실수, 문자열 자료형의 값을 다른 자료형으로 변환하기
 - `int()` 함수
 - 실수 값을 정수로 변환 (버림)
 - 정수 형태의 문자열을 정수로 변환
 - `float()` 함수
 - 정수 값을 실수로 형 변환
 - 숫자 형태의 문자열을 실수로 형 변환
 - `str()` 함수
 - 정수 값을 문자열로 형 변환
 - 실수 값을 문자열로 형 변환

자료형 변환

- `int()` 내장 함수

- 실수, 문자열 → 정수로 변환하기
- 실수 값을 정수로 변환
- 정수 형태의 문자열을 정수로 변환
- 실수 형태의 문자열은 정수로 변환할 수 없다. **error!!**

```
>>> c = int( '100' )
>>> c
100
>>> int( '100' )
100
>>> age = '21'
>>> 2022 - int(age) + 1
2002
>>> a = int(5.6)
>>> a
5
>>>
```

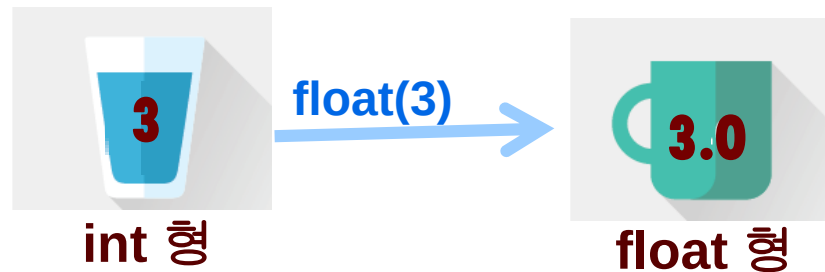
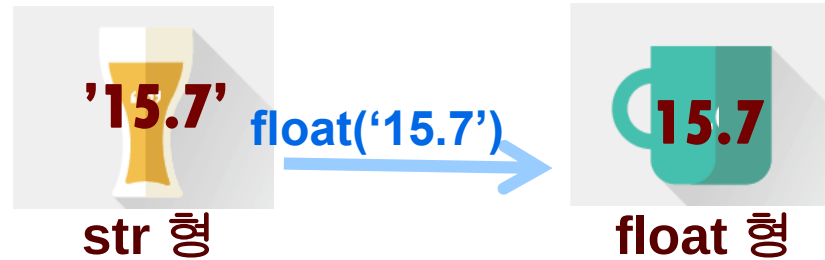


(출처: 컴퓨팅 사고력을 키우는 SW교육 파이썬)

자료형 변환

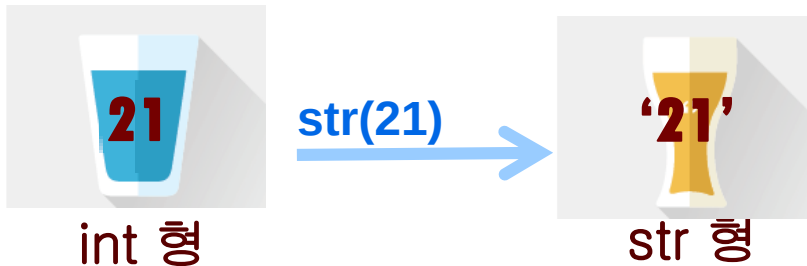
- `float()` 내장 함수
 - 정수, 문자열 → 실수로 변환하기
 - 정수 값을 실수로 변환
 - 숫자 형태의 문자열을 실수로 변환.

```
>>> z = '15.7'
>>> z
'15.7'
>>> z = float(z)
>>> z
15.7
>>> x = float('3')
>>> x
3.0
>>> float(3)
3.0
>>>
```



자료형 변환

- `str()` 내장 함수
 - 숫자 → 문자열로 변환하기
 - 정수 값을 문자열로 변환
 - 실수 값을 문자열로 변환



```
>>> age=21
>>> print( age, '살')
21 살   #값과 값 사이에 공백 출력
>>> str(age)
'21'
>>> print( str(age) + '살') #값과 값 연결. 공백없이 출력
21살
>>>
```



문자열과 숫자를 합칠 수 없다.

```
>>> print( 21 + '살이다.')
Traceback (most recent call last):
File "<pyshell#1>", line 1, in <module>
print( 21 + '살이다.')
TypeError: Can't convert 'int' object to str implicitly
```